

Problem Set 6 — STAT 380

Due Monday, March 23 at 11:59 p.m.

The 9 hour grace period does apply to this problem set. Therefore, you can submit before Tuesday, Mar. 24 at 8:59 a.m. without any late penalty. After that point, there is late penalty (see syllabus).

You make work with others. If you work with others, please make sure you include whom you worked with. You are responsible for writing your own solutions and code.

You may use the help documentation in [R](#), and you may use the internet if you need it. If you use the internet (including gen AI tools), you must cite the websites you used by including the URL.

See the course's generative artificial intelligence policy in the syllabus for more details about what gen AI use is acceptable.

Instructions

For this problem set, please answer the questions in a Quarto document. Your work must be organized (using headers and with work for questions appearing in order) and you must include BOTH CODE AND OUTPUT.

On Canvas, you will upload 2 files: a PDF document with your answers and a txt file of your code.

Remember you can add this code chunk to your Quarto document to save your code as a txt file.

```
```${r}
#| include: false
courseTools::saveCode(last_name = '') # add your last name in the quotes
```
```

Feel free to email me with questions or issues with your code.

(The questions begin on the next page.)

Question 1 is regarding LASSO regression, which we covered on 27 Feb.

Question 1

`MLB.csv` contains information about a sample of Major League Baseball (MLB) players from the 1986 and 1987 seasons. Variables include:

- `PlayerName`-Player's name
- `AtBat`-Number of times at bat in 1986
- `Hits`-Number of hits in 1986
- `HmRun`-Number of home runs in 1986
- `Runs`-Number of runs in 1986
- `RBI`-Number of runs batted in in 1986
- `Walks`-Number of walks in 1986
- `Years`-Number of years in the major leagues
- `CAtBat`-Number of times at bat during his career
- `Chits`-Number of hits during his career
- `CHmRun`-Number of home runs during his career
- `CRuns`-Number of runs during his career
- `CRBI`-Number of runs batted in during his career
- `CWalks`-Number of walks during his career
- `League`-factor with levels A (for American) and N (for National) indicating player's league at the end of 1986
- `Division`-factor with levels E (for East) and W (for West) indicating player's division at the end of 1986
- `PutOuts`-Number of put outs in 1986
- `Assists`-Number of assists in 1986
- `Errors`-Number of errors in 1986
- `Salary`-1987 annual salary on opening day in thousands of dollars
- `NewLeague`-factor with levels A (for American) and N (for National) indicating player's league at the beginning of 1987

Our goal is to predict a baseball player's 1987 salary on the basis of the other variables in the dataset (excluding player name).

Part (a).

Remove the `PlayerName` column and any players with a missing salary. Note: After this, `MLB` should have 263 observations of 20 variables.

Part (b).

Recall that, in order to use the `glmnet()` and `cv.glmnet()` functions for building LASSO regression models, we need to separate the input matrix and response variable vector.

Please do this now:

- Create the input matrix as we did in the notes using the `model.matrix()` function.
- Extract the response vector.

Part (c).

Using a seed of `52`, perform an 85/15 training/validation set split.

Part (d).

Using a seed of `2020` and 10-fold CV on the training data, use the `cv.glmnet()` function to find the optimal values of λ using both the minimum value of MSE rule and the 1-standard error rule.

For each of these 2 optimal values of λ ,

1. Report the `lambda` value.
2. Print out what variables are selected if we use this `lambda` choice in LASSO regression.

Part (e).

For each of the optimal values of λ that you found, compute the RMSE on the validation set data.

Part (f).

Perform backward elimination to select the “best” subset of predictors for predicting salary. Choose AIC or BIC, and justify your choice. Which variables remain in the model?

Part (g).

For the model selected by backward elimination, compute the RMSE on the validation set data.

Part (h).

Based on the RMSE for the validation data, which model performed the best? (Options are: `lambda.min`, `lambda.1se`, or backward elimination).

Question 2

Insert the following code chunk into your document to generate the data set `df` for this problem.

```
```\r\n# — Data generation —————\r\nset.seed(16802)\r\nn <- 250\r\n\r\nx <- runif(n, min = -5, max = 15)\r\n\r\ny <- ifelse(\r\n  x < 0,\r\n  20 * sin(x) + rnorm(n, 0, 2),\r\n  ifelse(\r\n    x < 7,\r\n    -0.5 * x^2 + 3.5 * x + 5 + rnorm(n, 0, 3),\r\n    ifelse(\r\n      x < 9,\r\n      rnorm(n, 10, 8),\r\n      -3 * (x - 9) + sample(c(-8, 8), n, replace = TRUE,\r\n                           prob = c(0.5, 0.5)) + rnorm(n, 0, 2)\r\n    )\r\n  )\r\n)\r\n\r\ndf <- data.frame(x = x, y = y)\r\n```\r\n
```

### Part (a).

Create a scatterplot of `x` and `y` from the `df` data frame.

### Part (b).

Fit a SLR model for `y ~ x`. Save the `lm()` object as `lm_fit`. Report the slope, intercept, and  $R^2$ .

### Part (c).

**Read the instructions carefully. You are writing your own kNN function here — not using the package we discussed in class.**

Write a function `knn_predict()` from scratch. It should take a new  $x$  value, the  $x$  and  $y$  vectors, and  $k$ , and then will return the mean  $y$  among the  $k$  nearest neighbors.

I provide a framework for how to do this below:

```
knn_predict <- function(x_new, x, y, k) {
 # Step 1: Compute the distance from x_new to every x point

 # Step 2: Find the indices of the k smallest distances

 # Step 3: Return the mean of the y values at those indices

}
```

### Part (d).

Use  $k = 10$ .

Now generate predictions across a fine grid of  $x$  values using this code.

```
x_grid <- seq(min(df$x), max(df$x), length.out = 300)

knn_preds <- sapply(x_grid, knn_predict,
 x = df$x,
 y = df$y,
 k = 10)
```

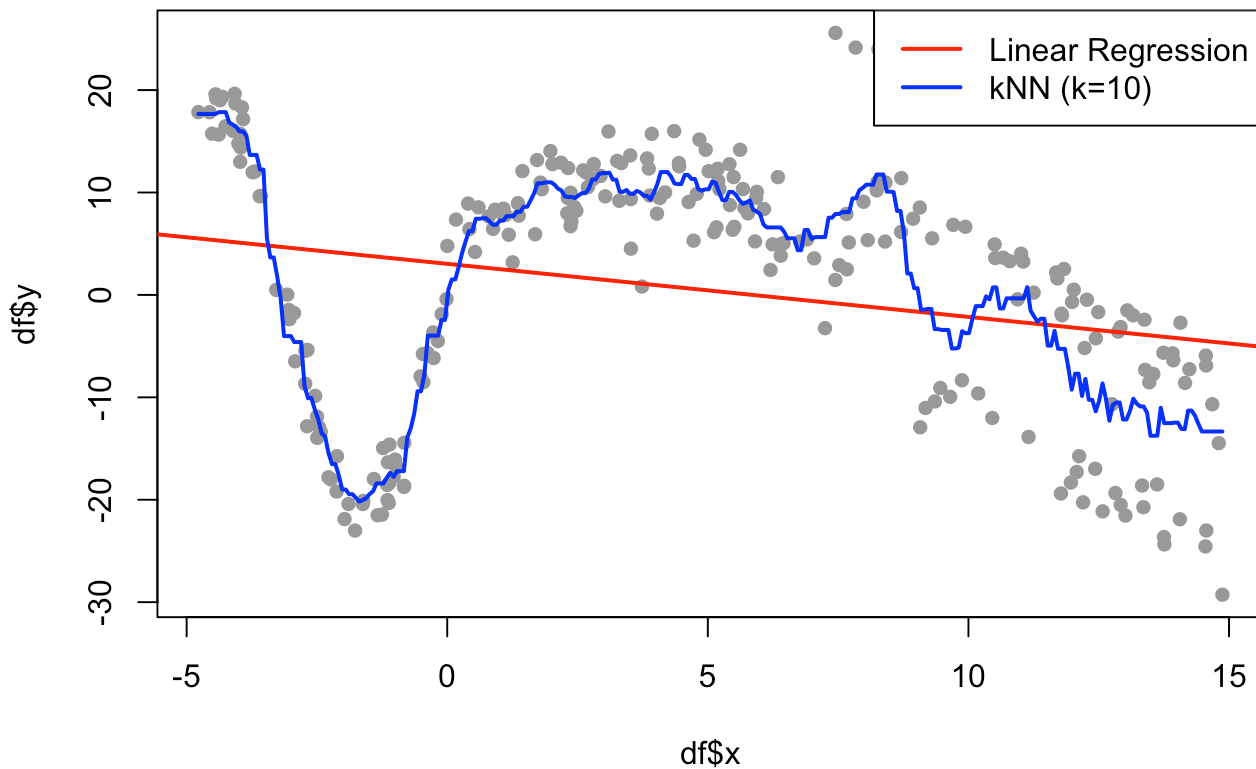
### Part (e).

Now reproduce the scatterplot from Part (a), and overlay the fitted curves:

- The linear regression line in red, and
- The kNN regression curve in blue.

Include a legend identifying each line.

(I have included what your plot should look like on the next page. Although this was done in base R plotting; if you use `ggplot`, it will look different and that is fine.)

**Part (f).**

1. Where does the linear regression model perform most poorly relative to the kNN curve?
2. kNN is a non-parametric method, while linear regression is parametric. What does this distinction mean, and what is one practical trade-off between the two approaches?
3. If you decreased  $k$  from 10 to 1, what would you expect the kNN curve to look like? What statistical problem does a very small  $k$  risk introducing?

## Question 3

---

The file NOVA.csv contains a sample of properties in the Northern Virginia region (Arlington County, Fairfax County, and so on) listed on Remax.com in April 2019. The data set contains the following variables:

- Bedrooms - number of bedrooms
- Bathrooms - number of bathrooms
- LivingSQFT - square footage of living area
- Age - age of the house in years
- DistToWH - distance from house to the White House in miles
- LotSize - lot size in acres
- ListingType - type of property (Condo/Townhome OR Single Family)
- Floors - the number of floors in the house
- Garage - the number of garage stalls
- Price - the price of the home in dollars

**The goal of this analysis is to predict the logprice of a property using the variables listed above. (Note we are not using the CountySchool variable.)**

### Part (a).

Create a variable called `logPrice` that represents the natural log of the house's price. We will use this going forward.

### Part (b).

Create a dummy variable for `ListingType` that takes value 1 when `ListingType` is SingleFamily and 0 otherwise. Name this variable `SingleFamHome`. We will use `SingleFamHome` instead of `ListingType` going forward.

### Part (c).

Standardize the inputs so that each has a mean of 0 and a standard deviation of 1. Moving forward, these will be called the scaled inputs. NOTE: Recall that the inputs are the x variables (INCLUDING indicators) and do not include the response (y) variable. Do NOT scale the response.

### Part (d).

Perform a 75/25 training/validation set split using seed `380`.

### Part (e).

Build a MLR model on the training data for predicting `logPrice` that includes all the scaled inputs.

Then, calculate the RMSE of the predictions for the "new" data (i.e., the validation set).

**Part (f).**

Build a kNN regression model for predicting `logPrice` using the scaled inputs. For each value of `k` from 1 to 50, build the model and store the RMSE. You should use a for loop to do this as we did in class. Display the RMSE values in your R output.

**Part (g).**

Create a line plot showing the RMSE as a function of  $k$ . Make the line red.

**Part (h).**

Based on the plot and the results of the for loop, which value of  $k$  should you use? Explain your choice.

**Part (i).**

Using the value of  $k$  selected in the last problem, calculate the RMSE of the predictions for the "new" data.

**Part (j).**

If we are interested in the ability to generalize to new data, which model (linear regression model or kNN with the optimal value of  $k$ ) would you choose based on this analysis? Explain.